

Softhouse Coding Test

Overview

Welcome! In this coding test you will build a small **command-line sensor data analyzer** in C or C++. The program reads temperature sensor readings from a CSV file and produces a summary report. Focus on writing clean, correct, and well-structured code.

Requirements

Input

The program reads a CSV file specified as a command-line argument. The CSV file has the following format:

```
None
timestamp,sensor_id,temperature
2025-01-15 08:00:00,sensor_1,21.5
2025-01-15 08:00:00,sensor_2,19.3
...
```

- **timestamp** — date and time in `YYYY-MM-DD HH:MM:SS` format
- **sensor_id** — a string identifier for the sensor (e.g. `sensor_1`)
- **temperature** — a floating-point value in degrees Celsius

The first line is always a header and should be skipped.

Output

The program shall print a report to `stdout` containing the following for **each sensor** found in the file:

1. **Sensor ID**
2. **Number of readings**
3. **Average temperature** (one decimal place)
4. **Minimum temperature** (one decimal place)
5. **Maximum temperature** (one decimal place)
6. A list of **anomalies** — any reading where the temperature deviates more than **± 10.0 °C** from that sensor's average.

7. For each sensor, verify that the timestamps appear in **non-decreasing order**. If a reading has a timestamp earlier than the previous reading for the same sensor, report it as an anomaly.

Example output

```
None
Sensor: sensor_1
  Readings : 4
  Average  : 22.1
  Min      : 20.5
  Max      : 23.8
  Anomalies: none

Sensor: sensor_2
  Readings : 4
  Average  : 18.5
  Min      : 17.0
  Max      : 19.8
  Anomalies:
    2025-01-15 14:00:00: 35.2
```

Error handling

- If no filename is given, print a usage message to `stderr` and exit with a non-zero exit code.
- If the file cannot be opened, print an error message to `stderr` and exit with a non-zero exit code.
- Lines that cannot be parsed should be skipped (with an optional warning to `stderr`).

Constraints

- Use **C** (C99 or later). You may also use C++ if you prefer.
- Use only the **C standard library** (or C++ standard library). No external libraries.
- The code shall be platform independent.

Some short sample data are in the end of this document.

What we look for

- **Correctness** — Does the program produce the right output?
- **Code clarity** — Is the code readable and well-organized?
- **Error handling** — Does the program handle bad input gracefully?
- **Resource management** — Are files closed and memory freed?

Good luck!

Sample data

Clean data (no anomalies)

None

```
timestamp,sensor_id,temperature
2025-01-15 08:00:00,sensor_1,21.5
2025-01-15 08:00:00,sensor_2,18.3
2025-01-15 10:00:00,sensor_1,22.3
2025-01-15 10:00:00,sensor_2,19.1
2025-01-15 12:00:00,sensor_1,23.8
2025-01-15 12:00:00,sensor_2,19.8
2025-01-15 14:00:00,sensor_1,20.5
2025-01-15 14:00:00,sensor_2,17.0
```

Data with anomalies

None

```
timestamp,sensor_id,temperature
2025-01-15 08:00:00,sensor_1,21.5
2025-01-15 08:00:00,sensor_2,18.3
2025-01-15 10:00:00,sensor_1,22.3
2025-01-15 10:00:00,sensor_2,19.1
2025-01-15 09:00:00,sensor_1,20.8
2025-01-15 12:00:00,sensor_1,23.8
2025-01-15 12:00:00,sensor_2,35.2
2025-01-15 14:00:00,sensor_1,20.5
2025-01-15 14:00:00,sensor_2,17.0
```